QA Automation Cheat Sheet

Cypress • Playwright • JUnit 5 • Mocha/Chai • Jenkins • POM • JaCoCo • Postman • Data-Driven

Muhammad Faizan | Interview Prep 2026

1. CYPRESS

What it is

JavaScript end-to-end testing framework. Runs INSIDE the browser — no WebDriver needed. Auto-waits for elements.

Key Facts

- JS/TS only
- Auto-wait — no Thread.sleep()
- Built-in assertions
- Screenshots + video on fail
- cy.intercept() for network stubs

Cypress vs Selenium

Cypress: faster, no WebDriver, JS only, auto-wait
Selenium: multi-language, multi-browser, older, needs explicit waits

Common Commands

```
cy.visit('/login')
cy.get('#user').type('faizan')
cy.get('.btn').click()
cy.url().should('include','/dash')
cy.contains('Welcome')
```

Install

```
npm install cypress --save-dev
npx cypress open
```

Page Object Model in Cypress

```
// pages/LoginPage.js
class LoginPage {
  visit()              { cy.visit('/login') }
  fillUser(u)          { cy.get('#username').type(u) }
  fillPass(p)          { cy.get('#password').type(p) }
  submit()             { cy.get('.btn-login').click() }
  assertDashboard()    { cy.url().should('include', '/dashboard') }
}
export default new LoginPage()

// tests/login.cy.js
import Login from '../pages/LoginPage'
describe('Auth', () => {
  it('valid login', () => {
    Login.visit()
    Login.fillUser('faizan')
    Login.fillPass('pass123')
    Login.submit()
    Login.assertDashboard()
  })
})
```

REST API Testing in Cypress

```
cy.request({
  method: 'POST',
  url: '/api/auth/login/',
  body: { username: 'faizan', password: 'pass123' }
}).then((res) => {
  expect(res.status).to.eq(200)
  expect(res.body).to.have.property('access')
})
```

2. PLAYWRIGHT

What it is
Microsoft's browser automation tool. Supports Chromium, Firefox, WebKit. Faster than Selenium, multi-browser unlike Cypress.

Key Facts

- Supports JS, Python, Java, C#
- Auto-wait built-in
- Multi-browser (Cypress is JS only)
- page.screenshot(), page.pdf()
- Network interception built-in

Playwright vs Cypress
Playwright: multi-browser, multi-language, better for cross-browser testing

Cypress: easier setup, better DX, JS only, single browser per run

Install

```
npm init playwright@latest
```

Basic Test

```
const { test, expect } = require('@playwright/test')

test('login flow', async ({ page }) => {
  await page.goto('http://localhost:3000/login')
  await page.fill('#username', 'faizan')
  await page.fill('#password', 'pass123')
  await page.click('.btn-login')
  await expect(page).toHaveURL(/dashboard/)
  await expect(page.locator('h1')).toContainText('Dashboard')
})
```

3. JUNIT 5

Key Annotations

```
@Test            // marks test
@BeforeEach      // before every test
@AfterEach       // after every test
@BeforeAll       // once before all
@AfterAll        // once after all
@DisplayName     // custom name
@Disabled        // skip test
@ParameterizedTest // data-driven
```

Core Assertions

```
assertEquals(expected, actual)
assertTrue(condition)
assertFalse(condition)
assertNotNull(object)
assertNull(object)
assertThrows(Ex.class, () -> fn())
assertAll(() -> assert1, () -> assert2)
```

Parameterized / Data-Driven Test

```
// testdata.csv:  username,password,expected
//                faizan,pass123,SUCCESS
//                admin,,FAIL

@ParameterizedTest
@CsvFileSource(resources = "/testdata.csv", numLinesToSkip = 1)
@DisplayName("Login data-driven test")
void testLogin(String username, String password, String expected) {
    String result = authService.login(username, password);
    assertEquals(expected, result);
}
```

```
// ValueSource example
@ParameterizedTest
@ValueSource(strings = {"", "  ", "null"})
void testEmptyUsername(String username) {
    assertThrows(IllegalArgumentException.class,
        () -> authService.login(username, "pass"));
}
```

JUnit 4 vs JUnit 5 — Key Differences

Feature	JUnit 4	JUnit 5
Annotation	@Before/@After	@BeforeEach/@AfterEach
Runner	@RunWith	@ExtendWith
Parameterized	@Parameterized (verbose)	@ParameterizedTest (native)
Architecture	Single jar	3 modules: Platform, Jupiter, Vintage

4. MOCHA + CHAI

What they are

Mocha = JS test runner (describes structure, hooks, async support)

Chai = assertion library (what you actually check)

Chai Assertion Styles

```
// expect (most common)
expect(val).to.equal(5)
expect(arr).to.have.length(3)
expect(obj).to.have.property('id')
expect(str).to.include('faizan')
expect(fn).to.throw(Error)

// assert
assert.equal(actual, expected)
assert.isTrue(condition)
```

Hooks

```
before(()  => {})   // once before suite
after(()   => {})   // once after suite
beforeEach(()=>{})  // before each test
afterEach( ()=>{})  // after each test
```

Full API Test Example

```
const { expect } = require('chai')
const request     = require('supertest')
const app         = require('../app')

describe('Auth API', () => {
  it('POST /login - valid credentials returns 200 + token', async () => {
    const res = await request(app)
      .post('/api/login')
      .send({ username: 'faizan', password: 'pass123' })
    expect(res.status).to.equal(200)
    expect(res.body).to.have.property('token')
  })

  it('POST /login - wrong password returns 401', async () => {
    const res = await request(app)
      .post('/api/login')
      .send({ username: 'faizan', password: 'wrong' })
    expect(res.status).to.equal(401)
  })

  it('POST /login - empty username returns 400', async () => {
```

```
    const res = await request(app)
      .post('/api/login')
      .send({ username: '', password: 'pass' })
    expect(res.status).to.equal(400)
  })
})
```

5. JENKINS / CI-CD

What is CI/CD

CI = Continuous Integration — auto build + test on every push
CD = Continuous Delivery/Deployment — auto deploy after tests pass

Why it matters for QA

- Tests run automatically on every commit
- Bugs caught early before production
- HTML reports generated per build
- No manual test execution needed

Jenkins Key Concepts

- Freestyle Project — UI-based, simple
- Pipeline — code-based via Jenkinsfile

- Build Trigger — what starts the pipeline
- Stage — a step in the pipeline
- Agent — machine that runs the job
- Artifact — output file saved from build

Jenkinsfile — Node.js Test Pipeline

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/Faizanyousaf140/node-auth-app' }
    }
    stage('Install') {
      steps { sh 'npm install' }
    }
    stage('Test') {
      steps { sh 'npm test -- --reporter mochawesome' }
    }
    stage('Report') {
      steps {
        publishHTML([
          reportDir:   'mochawesome-report',
          reportFiles: 'mochawesome.html',
          reportName:  'Test Report'
        ])
      }
    }
  }
  post {
    failure { echo 'Tests failed — notify team' }
    success { echo 'All tests passed!' }
  }
}
```

GitHub Actions Alternative

```
# .github/workflows/test.yml
name: Run Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: { node-version: '18' }
      - run: npm install
      - run: npm test
```

6. PAGE OBJECT MODEL (POM)

Design pattern that separates UI element locators from test logic. Each page of the app has its own class.

Folder Structure

```
tests/
  auth.test.js      <- test logic only
  payments.test.js
pages/
  LoginPage.js      <- selectors + actions
  DashboardPage.js
  PaymentPage.js
```

POM Class — Selenium Java

```
public class LoginPage {
  WebDriver driver;

  By usernameField = By.id("user");
  By passwordField = By.id("pass");
  By loginBtn      = By.cssSelector(".btn");

  public LoginPage(WebDriver d) { this.driver = d;
}

  public void enterUser(String u) {
    driver.findElement(usernameField).sendKeys(u);
  }
  public void enterPass(String p) {
    driver.findElement(passwordField).sendKeys(p);
  }
  public void clickLogin() {
    driver.findElement(loginBtn).click();
  }
}
```

Why POM

- If UI changes, update ONE page class only
- Tests are readable and short
- No duplicated selectors across tests
- Easy to scale to 100+ tests

7. JACOCO — CODE COVERAGE

What it is

Java code coverage tool. Integrated with Maven. Shows which lines/branches your tests actually executed.

pom.xml Setup

```
  org.jacoco
  jacoco-maven-plugin
  0.8.8


      prepare-agent


      report
      test
      report
```

Coverage Types

- Line coverage — lines executed
- Branch coverage — if/else paths taken
- Method coverage — methods called
- Class coverage — classes loaded

Run

```
mvn test jacoco:report
# Report: target/site/jacoco/index.html
```

Your number: 84%+ — say this confidently

8. API TESTING — POSTMAN

HTTP Methods

What to Validate in API Testing

```
GET    -> read data
POST   -> create data
PUT    -> update (full replace)
PATCH  -> update (partial)
DELETE -> delete
```

Status Codes to Know

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
422 Validation Error
500 Server Error
```

- Status code

- Response body — correct fields and values
- Schema — correct data types
- Headers — Content-Type, Auth token

- Response time — under acceptable limit
- Error messages — meaningful and correct

Postman Test Script

```
pm.test('Status 200', () => {
  pm.response.to.have.status(200)
})
pm.test('Has token', () => {
  const j = pm.response.json()
  pm.expect(j.token).to.not.be.undefined
})
```

REST API Testing with Supertest (Node.js)

```
const request = require('supertest')
const app     = require('../app')

// GET endpoint
it('GET /api/properties returns 200', async () => {
  const res = await request(app)
    .get('/api/properties/')
    .set('Authorization', `Bearer ${token}`)
  expect(res.status).to.equal(200)
  expect(res.body).to.be.an('array')
})

// POST endpoint
it('POST /api/properties creates property', async () => {
  const res = await request(app)
    .post('/api/properties/')
    .set('Authorization', `Bearer ${token}`)
    .send({ title: 'Flat 2B', city: 'Lahore', rent_amount: 25000 })
  expect(res.status).to.equal(201)
  expect(res.body).to.have.property('id')
})
```

9. DATA-DRIVEN TESTING

Running the same test with multiple input sets from an external source (CSV, Excel, JSON). Avoids copy-pasting test cases.

JUnit 5 — CSV Data Source

```
// src/test/resources/logindata.csv
// username,password,expected
// faizan,pass123,SUCCESS
// admin,,FAIL
// ,pass,FAIL

@ParameterizedTest
@CsvFileSource(resources = "/logindata.csv", numLinesToSkip = 1)
void loginTest(String user, String pass, String expected) {
```

```
    String result = authService.login(user, pass);
    assertEquals(expected, result);
}
```

JUnit 5 — Excel with Apache POI

```
FileInputStream fis  = new FileInputStream("testdata.xlsx");
Workbook wb          = new XSSFWorkbook(fis);
Sheet sheet          = wb.getSheetAt(0);

for (Row row : sheet) {
    if (row.getRowNum() == 0) continue;  // skip header
    String user     = row.getCell(0).getStringCellValue();
    String pass     = row.getCell(1).getStringCellValue();
    String expected = row.getCell(2).getStringCellValue();
    assertEquals(expected, authService.login(user, pass));
}
```

Cypress — Data-Driven with JSON

```
// cypress/fixtures/users.json
// [{"user":"faizan","pass":"pass123","expected":200},
//  {"user":"","pass":"pass","expected":400}]

import users from '../fixtures/users.json'

describe('Data-driven login', () => {
  users.forEach(({ user, pass, expected }) => {
    it(`Login: ${user} expects ${expected}`, () => {
      cy.request({ method:'POST', url:'/api/login',
                   body:{username:user, password:pass},
                   failOnStatusCode: false })
        .then(res => { expect(res.status).to.eq(expected) })
    })
  })
})
```

10. QUICK REFERENCE — ONE LINERS

Topic	One-liner to say in interview
Cypress	JS E2E framework, runs in browser, auto-waits, no WebDriver
Playwright	Microsoft tool, multi-browser, auto-wait, faster than Selenium
Selenium	Multi-language browser automation using WebDriver protocol
JUnit 5	Java test framework with @ParameterizedTest for data-driven testing
Mocha	JS test runner — describe/it structure, async support, hooks
Chai	JS assertion library — expect().to.equal() style
POM	Design pattern: page classes hold selectors, tests hold logic
JaCoCo	Java code coverage tool — I achieved 84%+ in my framework
Jenkins	CI server — runs tests automatically on every code push
Data-driven	Same test, multiple CSV/Excel inputs — no copy-paste test cases

Regression	Re-run existing tests after changes to catch regressions
API Testing	Validate status code, body, schema, headers, response time
POM (Postman)	Not same as Page Object — it's Package Object Model, the npm term

11. SAY THESE — LAST 10 MINS BEFORE INTERVIEW

" I used Cypress with Page Object Model for RentEase end-to-end testing — covering auth, property listing, and payment flows. "

" JUnit 5 parameterized tests reading from CSV files — 84%+ JaCoCo code coverage. "

" Jenkins CI pipeline with automated Mocha/Chai execution and HTML report generation. "

" API testing using Postman and Supertest — status codes, response schema, and data consistency validation. "

" Page Object Model keeps selectors in page classes and test logic in test files — makes the suite maintainable at scale. "

" Data-driven testing means one test method handles 20 scenarios from a CSV — no duplication. "

" I studied Software Engineering and Software Design — POM is itself a design pattern, same principles. "